

AIDA vs A2A (Agent2Agent)

AIDA positioning · best-effort comparison — live source:
github.com/joemooney/aida

rendered 2026-07-10

Last updated: 2026-07-09

The TL;DR: **A2A and AIDA are not competitors — they operate on different layers, and A2A explicitly disclaims the layer AIDA occupies.** A2A is an **agent↔agent transport**: discovery, task delegation, message/artifact exchange between running agents. It is, by its own specification, **stateless** — agents coordinate “*without sharing internal memory, state, or tools.*” AIDA is the **durable, multi-vendor-readable coordination record** — the git-canonical spec graph, queue, briefs, mailbox, and code↔spec traces that *are* the shared state. A2A carries the live handoff; AIDA holds the record that survives it. The honest answer to “should I use AIDA or A2A?” is **both, different layers** — not pick one.

Today AIDA neither implements nor depends on A2A. This page explains why that’s the correct default, and where an interop surface *could* fit.

What A2A is

A2A (Agent2Agent) is an open protocol, launched by Google (~2025) with a large partner set and since donated to the **Linux Foundation** (the same home as MCP and AGENTS.md; IBM’s **ACP** merged into A2A in 2025). It standardizes how independently-built agents interoperate:

- **Agent Cards** — capability/endpoint discovery (an agent advertises what it can do).
- **Tasks / messages / artifacts** — a JSON-RPC-style exchange to delegate work and return results.
- **Framework-agnostic** — an agent on one vendor/stack can call an agent on another.

It is a genuine, well-backed standard, and it is complementary to MCP: **MCP is agent↔tool, A2A is agent↔agent.** If you need two live agents from different vendors to hand each other tasks over the wire, A2A is the emerging lingua franca and worth speaking.

Be precise — the load-bearing property is its statelessness. A2A deliberately does *not* standardize a shared, persistent record of what’s being worked, by whom, in what state. It coordinates *between running agents* and leaves durable state to each agent.

That is a reasonable scope for a transport standard — and it’s exactly the boundary that leaves AIDA’s slice open.

The layer distinction (the crux)

	A2A	AIDA
Layer	live agent↔agent transport	durable coordination record
State	explicitly stateless (“no shared memory/state/tools”)	is the shared, versioned state (git-canonical)
Liveness	both agents must be running + reachable	agents needn’t be simultaneously live; the record persists across restarts
Unit	a task/message/artifact in flight	a spec graph with stable IDs, typed relationships, code↔spec traces
Human role	not addressed	a human holds and audits the record

A2A explicitly *disclaims* the thing AIDA is. So the overlap is narrow — only the *message transport* — and the differentiation is orthogonal.

Why the gap persists (and why that’s durable)

This isn’t a temporary hole A2A will close in the next version — it’s a consequence of **incentive**, which is why it ages well. Interop standards bodies build the **floor and the guardrails** (transport, discovery, identity, safety) so the ecosystem can grow; they do *not* ship an opinionated, durable, multi-vendor-readable *coordination record*, because that would mean taking a position on how work is modeled — a product decision, not a standard. AIDA’s own market-landscape research states it plainly: “MCP (single-requestor), A2A (explicitly stateless), ACP (merged into A2A) — none standardizes a durable, multi-vendor-readable coordination record. The standards bodies are building the floor and the guardrails, not the record.”

A2A’s ascent is therefore **validation, not threat**: it confirms the premise that agents must coordinate across vendors, while explicitly leaving the record slice unclaimed.

Where they meet — the interop opportunity

AIDA’s stated posture is to **generate/speak** open protocols, not replace them (it already exposes MCP for the agent↔tool axis). The analogous move for A2A, *if and when warranted*:

- Expose an **A2A surface** so an A2A-native agent can deliver *into* AIDA’s substrate — an A2A message landing as a mailbox item or a brief pickup — without AIDA abandoning its record model.
- The record stays the source of truth; A2A becomes one more transport that feeds it, alongside the CLI, MCP, and plain git.

This is “meet the ecosystem,” not “adopt a dependency.”

Honest caveats

- **The “speak A2A” posture is stated, not built.** There is no A2A surface in AIDA today — this is a position, not a feature. Don’t claim otherwise.
- **A2A is real and well-backed.** Dismissing it would be a credibility error; the correct framing is complementary layers, not superiority.
- **Interop surfaces are optional; the substrate is primary.** This mirrors the MCP finding (MCP costs $\sim 2\times$ the token-efficient CLI for equal-or-better results, so the CLI is AIDA’s primary agent surface). An A2A surface would be additive interop, gated on real demand — not a core dependency.

Verdict — “should I use AIDA or A2A?”

Both, if you’re doing cross-vendor multi-agent work — they solve different problems. Use A2A (or MCP, or plain git) as the transport your agents speak. Use AIDA for the durable, queryable, human-holdable record of *what* is being worked, by whom, in what state, and how the code traces back to it. If you only need two live agents to hand each other a task, A2A alone is enough and AIDA is overkill. If you need that coordination to *survive*, be auditable, and stay consistent across vendors and over the project’s whole life, that’s the record A2A deliberately leaves to someone else — and that’s AIDA’s bet.

Related

- docs/research/2026-06-26-agent-coordination-market-landscape.md — “the standards bodies are building the floor and the guardrails, not the record.”
- docs/competitive-analysis/marketplace-roster.md — MCP/A2A/ACP standards row + the “durable coordination record” open-frontier gap.
- docs/positioning/composition.md — how AIDA composes with the tools it doesn’t replace.